

Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

IC-1801 Taller de Programación - Prof. Mauricio Avilés

Proyecto Programado – Autómatas celulares

Introducción

Los autómatas celulares son modelos que consisten en una cuadrícula de células, cada una con un conjunto de estados posibles y células vecinas (vecindario) que determinan su próximo estado. La cuadrícula de células avanza en el tiempo a través de una serie de iteraciones, a partir de un estado inicial predefinido. En cada iteración se determina el estado de cada célula a partir de una función que considera el estado propio y el de sus vecinos. Dependiendo de las reglas utilizadas y el estado inicial las células pueden llegar a formar patrones que persisten a través del tiempo y tienen comportamientos bien definidos.

El estudio de los autómatas celulares ha llevado a aplicaciones como simulaciones de tráfico, diseño de estructuras, composición musical, criptografía, generación de números aleatorios, vida artificial, explicar comportamientos en la naturaleza como el crecimiento de cristales, evolución, comportamiento de fluidos; entre otros.

Software a desarrollar

El proyecto consiste en la implementación de varios sistemas de autómatas celulares por medio del lenguaje de programación Python y alguna biblioteca gráfica como Pygame. Los sistemas a implementar son:

- Generalización del Juego de la vida de Conway (Life-Like)
- Generalización del autómata Hormiga de Langton

Para los autómatas celulares se desean desarrollar las siguientes características:

1. Solicitar parámetros iniciales al usuario por medio de interfaz gráfica (se recomienda usar easygui)
 - a. Cantidad de filas y columnas
 - b. Tamaño de las celdas
 - c. Parámetros específicos de cada autómata
2. Pausar y continuar con la tecla espacio.
3. Cambiar el estado de las células con el clic del mouse (con cada clic cambiar la célula al siguiente estado)
4. Guardar el estado completo, dimensiones, tamaño de la célula y parámetros del autómata en un archivo mediante la tecla G (se recomienda usar pickle)
5. Cargar el estado completo, dimensiones, tamaño de la célula y parámetros del autómata desde un archivo mediante la tecla C (pickle)
6. Reiniciar la matriz con valores aleatorios con la tecla R

7. Reiniciar la matriz con valores neutros con la tecla B (ceros o equivalente)

Autómata Life-Like

Los autómatas llamados “Life-Like” son generalizaciones del autómata del Juego de la Vida de Conway. Al igual que el famoso autómata de Conway, las células tienen únicamente dos estados, utilizan una cuadrícula bidimensional, los vecinos se cuentan usando el vecindario de Moore (las ocho celdas alrededor) y el nuevo estado de la célula puede expresarse como una función de su propio estado y el de su vecindario.

En el autómata de Conway, las reglas son las siguientes:

- Nacimiento: célula muerta con 3 vecinos vivos.
- Muerte: célula viva con menos de 2 vecinos o más de 3.
- Permanencia: cualquier otro caso la célula permanece en su mismo estado.

Si nos enfocamos en las células vivas, estas reglas pueden escribirse esta otra manera:

- Nacimiento: 3 vecinos vivos.
- Supervivencia: 2 o 3 vecinos
- En cualquier otro caso la célula muere o permanece muerta

Entonces, las reglas del autómata de Conway pueden expresarse como “B3/S23”, donde B=“birth” y S=“survival”. Es decir, usando esta notación, podemos leer las reglas como: las células nacen cuando tienen exactamente 3 vecinos y siguen vivas cuando tienen 2 o 3 vecinos vivos. En general, “Bx/Sy” sería la descripción de un autómata celular Life-Like donde las células nacen con la cantidad de vecinos en los dígitos que aparecen en el string x y sobreviven con los dígitos que aparecen en y.

El objetivo es escribir esta generalización del autómata, basándose en la implementación vista en clase. Al ejecutarse, el programa solicita al usuario los casos donde se producen nacimientos (dígitos de 0 a 8) y los casos donde las células sobreviven (también dígitos de 0 a 8).

Lineamientos para la implementación del autómata Life-Like

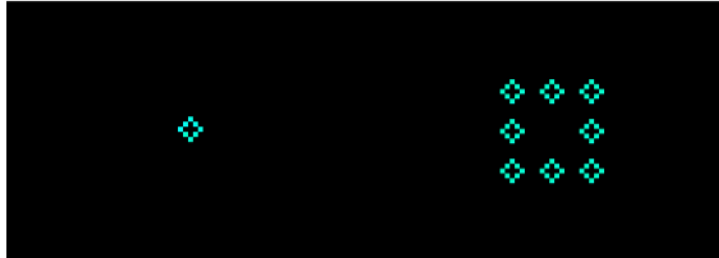
Debe reutilizarse y adaptarse el código del autómata de la vida de Conway que se desarrolló en clase, usando las siguientes subrutinas para su implementación:

1. `generar_matriz_aleatoria(...)`: crea una matriz de las dimensiones indicadas con valores aleatorios de 0 o 1.
2. `generar_matriz_vacia(...)`: crea una matriz de las dimensiones indicadas con valores en cero.
3. `transicion_celula(...)`: recibe el estado de la célula (0 o 1), una lista con el estado de los vecinos y las reglas a aplicar. Retorna como resultado el nuevo estado de la célula de acuerdo con las reglas.
4. `transicion(...)`: recibe la matriz del autómata y genera una nueva matriz que es el resultado de aplicar las reglas del autómata a todas sus células.

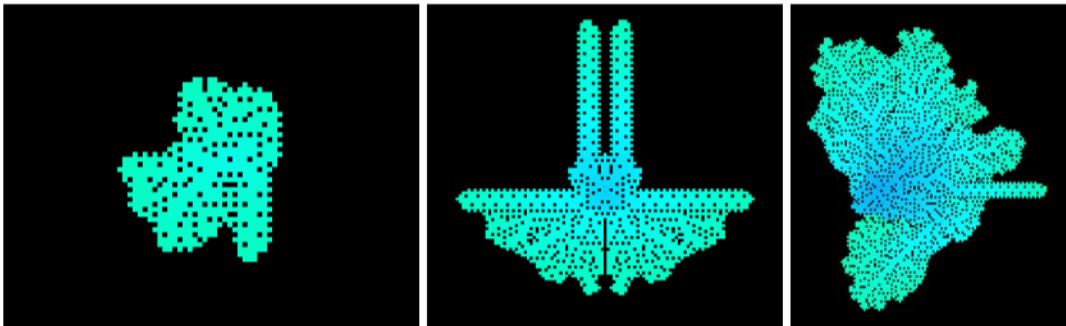
Pueden agregarse subrutinas adicionales, pero la estructura y organización general debe ser similar a la vista en clase.

Ejemplos de reglas Life-Like

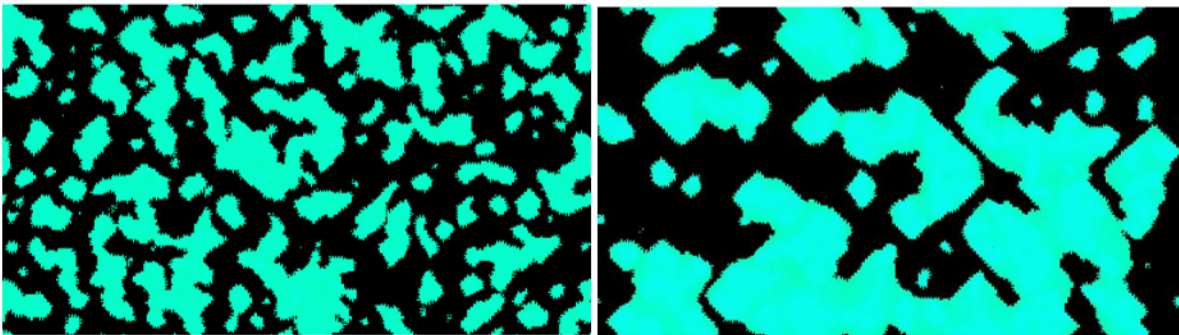
Replicator B1357/S1357: Al iniciar con un patrón de células vivas, luego de varias iteraciones el patrón es reemplazado por muchas copias de sí mismo.



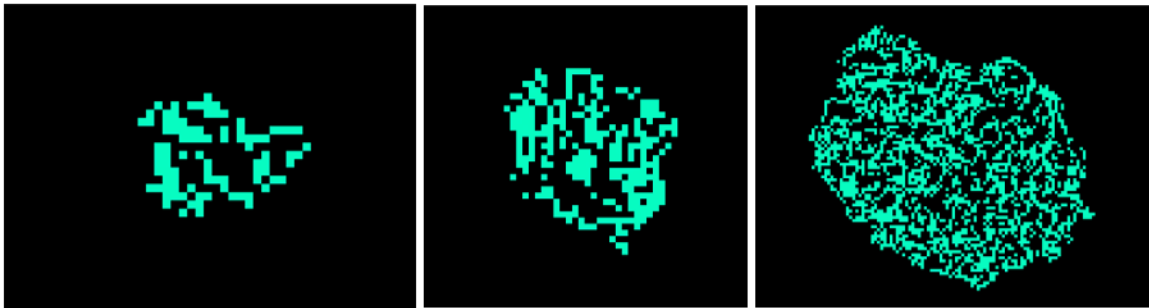
Life without death B3/S012345678: Similar al de Conway, pero las células nunca mueren. Las estructuras crecen en patrones tipo escalera.



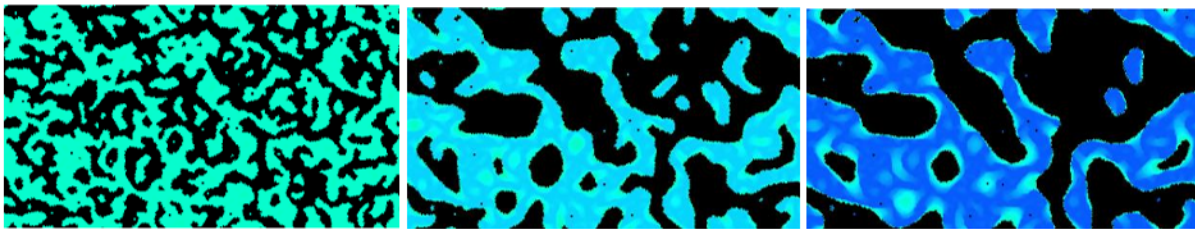
Diamoeba B35678/S5678: Forma diamantes grandes con bordes que fluctúan.



34 Life B34/S34: Tiene un comportamiento similar al de Conway pero los patrones tienen a explotar indefinidamente.



Anneal B4678/S35678: Se generan curvas que con el tiempo tienen a converger hacia la curva más corta.



¿Qué otras reglas con comportamientos interesantes existirán? ¡Hay que probar inventar algunas!

Generalización de la hormiga de Langton

El autómata de la hormiga de Langton tiene reglas muy simples, pero genera un comportamiento emergente complejo. Fue inventado por Chris Langton en 1986 y utiliza células con dos estados.

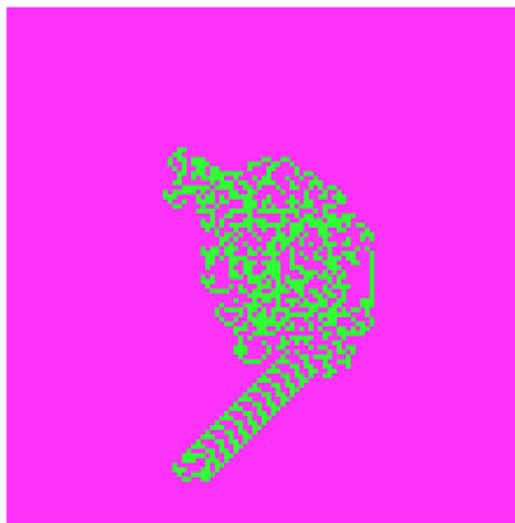
Utiliza una matriz bidimensional de células que tiene dos estados: blanco o negro. También, se utiliza una abstracción llamada “hormiga” que tiene una posición dentro la matriz y una dirección (arriba, abajo, izquierda, derecha).

Las reglas giran en torno al estado de la célula donde se encuentra la hormiga y la dirección que tiene:

1. Si se encuentra en una célula blanca, gira a la derecha 90 grados, cambia el color de la célula y avanza una célula.
2. Si se encuentra en una célula negra, gira a la izquierda 90 grados, cambia el color de la célula y avanza una célula.

Estas dos simples reglas llevan a un comportamiento complejo. Si se inicia la matriz en un solo color y se ubica la hormiga en su centro, se obtendrán tres tipos de comportamientos emergentes:

1. Simplicidad: durante los primeros cientos de pasos, la hormiga crea patrones muy simples que a menudo son simétricos.
2. Caos: después de cierto punto emerge un patrón irregular que continúa hasta cerca del paso 10000.
3. Orden emergente: finalmente, la hormiga empieza a construir una “autopista”, un patrón de 104 pasos que se repite indefinidamente.



En la imagen puede observarse el patrón desarrollado por la hormiga después de 10000 iteraciones, en la parte inferior puede verse la repetición que genera la “autopista” mencionada anteriormente.

Al generalizar las reglas de este autómata, podemos pensar que las células de la matriz, en lugar de dos colores, pueden tener N colores que cambian de manera cíclica, y por cada color la hormiga hace lo mismo que en la versión original, girar a la izquierda o a la derecha, cambiar el color de la célula al siguiente color

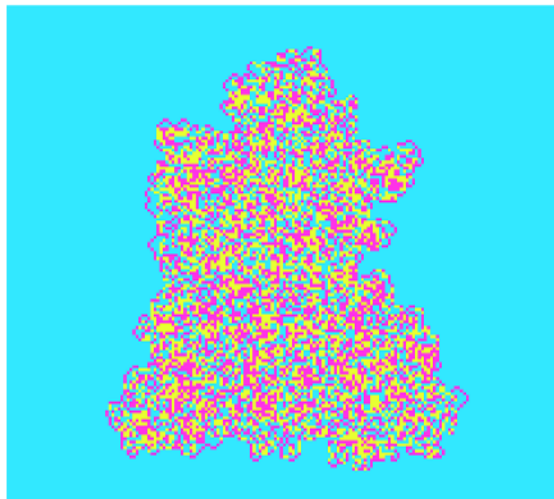
disponible y avanzar un espacio. Lo único que hay que especificar es en qué dirección debe hacer el giro la hormiga cuando se encuentra cada color.

De esta manera, el autómata original es la hormiga "LR". Es decir, gira a la izquierda con el primer color, y a la derecha con el segundo color.

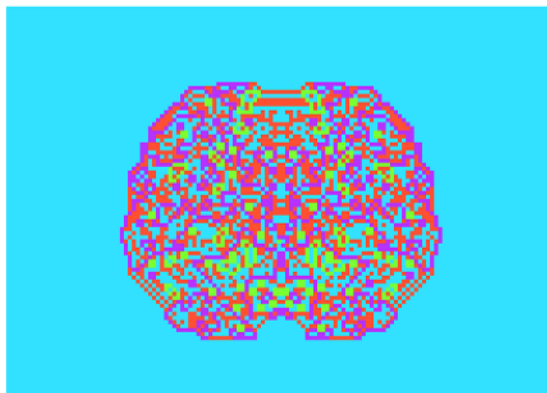
El objetivo es escribir esta generalización del autómata. Al ejecutarse, el programa solicita al usuario un string conformado por Ls y Rs cuyo largo indica la cantidad de colores a utilizar en las células y la acción que realiza la hormiga en cada uno de ellos.

Ejemplos de generalizaciones con múltiples colores

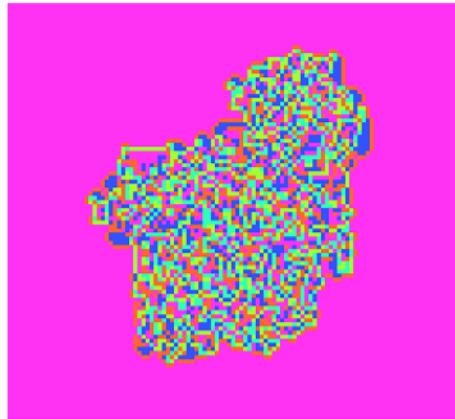
RLR: con tres colores, crece de forma caótica.



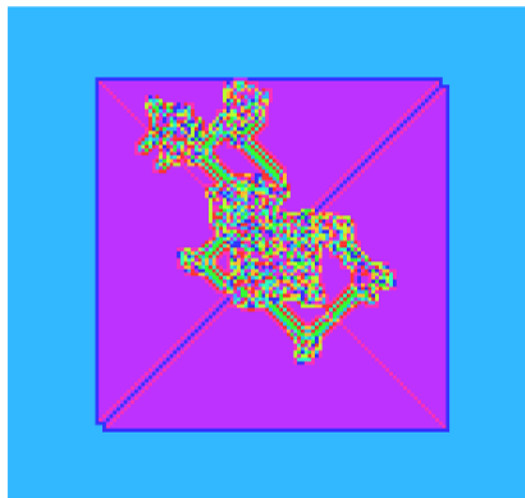
LLRR: con cuatro colores, genera patrones simétricos. En general cualquier tira que contenga pares de LL y RR consecutivos tiende a generar patrones simétricos.



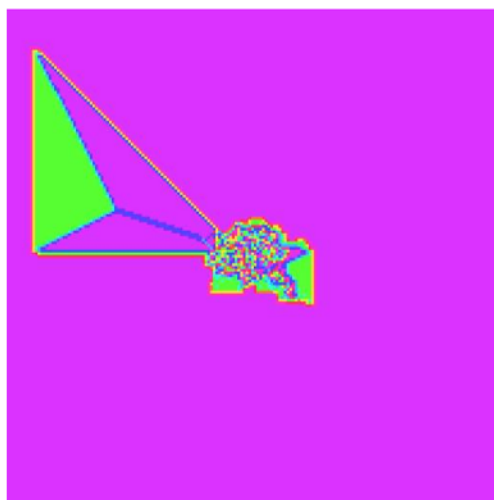
RLRLR: patrón caótico.



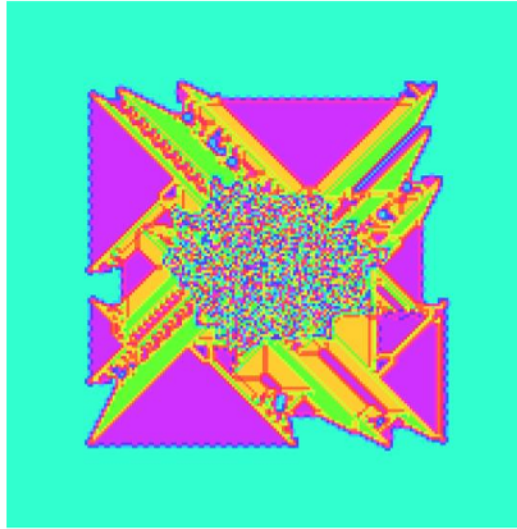
LRRRRLLR: crece en forma de cuadrado.



RLLLLLLLRRR: inicialmente caótico, llega a construir un triángulo que va creciendo.



LLRLLLLRRRRRR: caótico en el centro, estructuras triangulares que crecen alrededor.



¿Qué otros comportamientos interesantes estarán escondidos detrás de este autómatas?

Lineamientos para la implementación de la hormiga de Langton

Este autómatas es de una naturaleza muy distinta al anterior, no es necesario recorrer toda la matriz para calcular el siguiente estado del autómatas, basta con analizar la posición en la que se encuentra la hormiga, y luego actualizar la celda y el estado de la hormiga de acuerdo con las reglas.

1. Piense en cómo representar la hormiga, ya que esta tiene una posición dentro de la matriz y además una dirección de cuatro posibles.
2. Elabore una subrutina "girar_hormiga(direccion)" que le permita actualizar la abstracción hormiga de acuerdo con un giro hacia la izquierda o la derecha.
3. Elabore una subrutina "avanzar_hormiga()" que actualice la posición de la hormiga de acuerdo con su posición y dirección actual.
4. Elabore una subrutina "siguiente()" que se encargue de analizar el número de color de la celda donde se encuentra la hormiga y buscar en el string de entrada qué acción realizar para actualizar la matriz y la hormiga.
5. Escriba una subrutina "generar_colores(n)" que le permita obtener una lista de diferentes colores en formato RGB según la cantidad a utilizar en el autómatas.

Pueden agregarse subrutinas adicionales, pero debe seguirse la estructura aquí descrita.

Documentación

Toda subrutina debe llevar como documentación interna comentarios con lo siguiente:

- Descripción de la función
- Entradas
- Salidas
- Restricciones

En cuanto a la documentación externa, debe entregarse un manual de usuario en formato PDF que contenga los siguientes puntos:

1. Explicación del propósito de los programas.
2. Instrucciones para la instalación del programa y cualquier requisito que tenga. Con imágenes de ejemplo.
3. Instrucciones de utilización de todas las características del programa. Con imágenes de ejemplo.

Forma de trabajo

El proyecto será desarrollado en equipos de dos personas. No se aceptarán trabajos individuales, salvo previa autorización del profesor y con alguna justificación especial que lo amerite.

Entrega

El tiempo asignado para la tarea programada es de 2 semanas. Debe entregarse un archivo **archivo ZIP** que contenga dos cosas:

1. Un programa de Python por cada autómata celular.
2. Un archivo PDF con el manual de usuario

Evaluación

La tarea tiene un valor de 20% de la nota final, en el rubro de Proyectos Programados.

Desglose de la evaluación de la tarea programada:

Documentación:	20%
----------------	-----

Programación:	80%
---------------	-----

Recomendaciones adicionales

Pruebe cada función individualmente. No implemente todo el programa sin verificar el funcionamiento por separado de cada una de sus funciones. Esto dirige a errores que son más difíciles de encontrar.

Recuerde que el trabajo es en equipos, es indispensable la comunicación y la coordinación entre los miembros del subgrupo.

Ante cualquier duda que tenga consulte al profesor o al asistente del curso para seguir adelante.

Plagios no serán tolerados en ninguna circunstancia. Cualquier intento de fraude será evaluado con una nota de cero y se llevará a cabo el proceso estipulado en el Reglamento de Enseñanza-Aprendizaje del Instituto Tecnológico de Costa Rica. Siempre escriba su propio código.

Referencias

Cellular automaton. (2016, May 19). In *Wikipedia, The Free Encyclopedia*. Retrieved 00:19, May 27, 2016, from https://en.wikipedia.org/w/index.php?title=Cellular_automaton&oldid=720992591

Langton's ant. (2015, December 16). In *Wikipedia, The Free Encyclopedia*. Retrieved 00:20, May 27, 2016, from https://en.wikipedia.org/w/index.php?title=Langton%27s_ant&oldid=695460564

LifeWiki. (2025, June 9). https://conwaylife.com/wiki/Main_Page